

Rapport d'Avancement

Filière : Intelligence Artificielle et Cybersécurité

Système de Surveillance Intelligente par Caméra sur Raspberry Pi

Architecture logicielle et tableau de bord temps réel

Réalisé par : ALIOUALI Othmane, ELJARIDA Saad Eddine

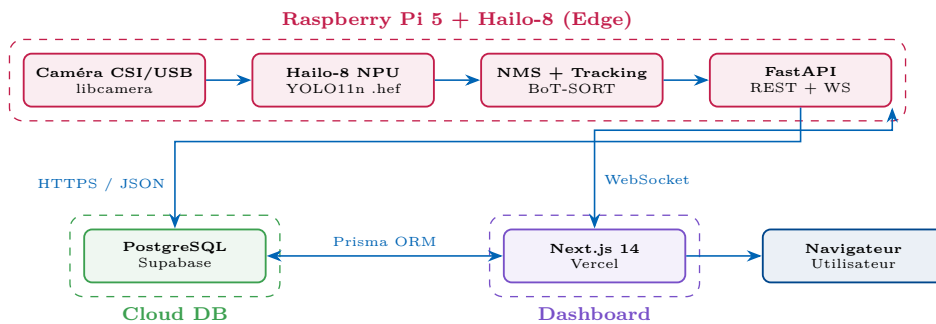
Encadré par : Pr. ABOUHILAL Abdelmoula

Phase 3 — Architecture et Dashboard

Semaine 4

Pipeline Edge-to-Cloud — Interface de supervision temps réel

† Les valeurs de latence présentées dans ce tableau sont des **estimations générées par Claude AI** à partir de données publiques et de la documentation officielle. Elles ne proviennent pas de mesures réelles sur notre plateforme et sont susceptibles de varier significativement selon la configuration matérielle, la charge CPU, la qualité du réseau et les conditions d'utilisation. Des benchmarks réels seront effectués lors de la phase d'implémentation.



Couche	Technologie	Rôle	Env.
Inférence IA	HailoRT + YOLO11n	Détection objets temps réel, 26 TOPS, ~30 FPS	RPi 5
Tracking	BoT-SORT / ByteTrack	Suivi multi-objets, assignation d'IDs uniques	RPi 5
API Backend	FastAPI (Python)	REST + WebSocket, streaming événements vers cloud	RPi 5
Base de données	Supabase (PostgreSQL)	Stockage détections/alertes, push auto vers le dashboard à chaque insertion	Cloud
Dashboard	Next.js 14 + shadcn/ui	SSR, composants : Live Stream, Alertes, Stats, Heatmap	Vercel
ORM / Graphiques	Prisma + Recharts	Accès type-safe PostgreSQL + visualisation données	Vercel

Étape	Composant	Données	Latence
1	Caméra → CPU	Frames 1080p, resize 640p, normalisation RGB	<10 ms
2	Hailo-8 NPU	YOLO11n HEF → bboxes + classes + scores	~5–8 ms
3	CPU (post-traitement)	NMS (seuil 0.45) + BoT-SORT + logique alertes	~3–5 ms
4	FastAPI → Cloud	JSON {detections, alerts} via HTTPS + WebSocket	~50–150 ms
5	Dashboard (navigateur)	Rendu React : stream live, alertes, graphiques	<100 ms
Latence totale end-to-end		Caméra → Alerte affichée sur Dashboard	<400 ms

▷ **Résumé : Edge** — tout le traitement IA reste local sur le RPi 5 (YOLO11n + BoT-SORT + FastAPI), seuls les métadonnées JSON sont envoyées au cloud. **Cloud** — Supabase (PostgreSQL + push automatique des nouvelles lignes vers le dashboard, sans polling) + Next.js 14 sur Vercel. **Latence cible** : détection → alerte affichée en <400 ms.